МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение

высшего образования

«МИРЭА — Российский технологический университет»

Институт кибербезопасности и цифровых технологий

Кафедра КБ-14 «Цифровые технологии обработки данных»

ОТЧЁТ

по практическим работам № 3–4

по дисциплине «Разработка мобильных приложений»

Тема: «Архитектура MVVM, ViewModel, LiveData, MediatorLiveData и списки на RecyclerView в Android-приложении Cybersport Tracker»

Выполнил: студент группы _____

_____ / _____

Проверил: _____

Москва, 2026

Содержание

Введение
1 Теоретическая часть
1.1 Архитектурные паттерны MVC, MVP, MVVM, MVI, VIPER
1.2 MVVM и чистая архитектура проекта
1.3 ViewModel: жизненный цикл, ViewModelProvider, Factory
1.4 LiveData и организация реактивного состояния
1.5 MediatorLiveData как средство объединения источников
1.6 RecyclerView: Adapter, ViewHolder, LayoutManager, DiffUtil
2 Практическая работа № 3. ViewModel, LiveData, MediatorLiveData
2.1 Контрольное задание и исходное состояние проекта
2.2 Экранные ViewModel и фабрики
2.3 Обновление интерфейса через LiveData
2.4 Объединение сетевых данных и Room с помощью MediatorLiveData
2.5 Результаты практической работы № 3
3 Практическая работа № 4. RecyclerView и передача данных через LiveData
3.1 Контрольное задание
3.2 Заглушка набора данных в слое data
3.3 Реализация списка команд на RecyclerView
3.4 Передача данных из ViewModel в адаптер
3.5 Состояния списка и результаты работы
4 Тестирование и проверка работоспособности
Заключение
Список использованных источников
Приложение А. Перечень изменённых файлов

Введение

Современное мобильное приложение, работающее с сетевыми данными, локальной базой данных и несколькими экранами, невозможно поддерживать, если вся логика сосредоточена в Activity. Поворот экрана, смена конфигурации, повторный вход на экран и обновление данных из базы должны обрабатываться предсказуемо. Именно этим задачам посвящены практические работы № 3 и № 4 дисциплины «Разработка мобильных приложений».

В качестве объекта разработки используется учебное Android-приложение Cybersport Tracker — киберспортивный трекер турниров, команд и матчей. Проект состоит из трёх Gradle-модулей: app (слой представления), domain (модели, интерфейсы репозиториев и сценарии использования) и data (реализации репозиториев, Room, имитация сетевого API, SharedPreferences, TFLite). Приложение написано на Java 11 с использованием AndroidX, Material 3 и Room.

Цель работ — закрепить навыки проектирования Android-приложения по шаблону MVVM, разделения ответственности между Activity и ViewModel, реактивного обновления интерфейса через LiveData, объединения нескольких источников данных с помощью MediatorLiveData и построения производительного списка на RecyclerView.

Для достижения цели были поставлены следующие задачи:

1. проанализировать архитектурные шаблоны MVC, MVP, MVVM, MVI и VIPER и обосновать выбор MVVM;
2. вынести взаимодействие Activity со слоем domain в экранные ViewModel;
3. организовать обновление состояния интерфейса исключительно через LiveData;
4. объединить замоканные данные «сетевого» слоя и наблюдаемые данные Room с помощью MediatorLiveData;
5. реализовать отображение списка команд с изображениями через RecyclerView;
6. проверить сборку, авторизацию, гостевой режим, избранное, поиск, фильтрацию и поворот экрана.

Практическая значимость работы состоит в том, что полученная архитектура пригодна для дальнейшего развития приложения: замены mock-источника на Retrofit, добавления новых экранов и покрытия ViewModel модульными тестами.

1 Теоретическая часть

1.1 Архитектурные паттерны MVC, MVP, MVVM, MVI, VIPER

Архитектурный шаблон задаёт правила разделения ответственности между пользовательским интерфейсом, логикой представления и бизнес-логикой. Чем последовательнее это разделение, тем проще тестировать код, менять источники данных и переиспользовать экраны. Рассмотрим пять распространённых подходов.

MVC (Model — View — Controller)

Model содержит данные и бизнес-логику, View отображает данные и принимает события пользователя, Controller связывает их: получает события от View, изменяет Model и обновляет представление. Шаблон прост и исторически популярен в веб-разработке, однако в больших приложениях контроллер перегружается, а View и Model оказываются связанными слишком тесно.

MVP (Model — View — Presenter)

В MVP Presenter управляет представлением вручную: View передаёт ему события, Presenter обращается к Model и вызывает у View методы отображения. View становится пассивным, а Presenter легко тестируется без Android-инфраструктуры. Недостаток — большое количество интерфейсов и риск «раздувания» Presenter при росте экрана.

MVVM (Model — View — ViewModel)

В MVVM View подписывается на ViewModel, а ViewModel хранит состояние экрана и не знает о конкретной View. В Android это реализуется через androidx.lifecycle.ViewModel и LiveData: Activity наблюдает за LiveData, а ViewModel переживает пересоздание Activity при повороте экрана. Двусторонняя привязка данных не обязательна — достаточно одностороннего потока «состояние → View», что хорошо ложится на Java и XML-разметку.

MVI (Model — View — Intent)

MVI строит приложение вокруг однонаправленного потока данных: View порождает Intent (намерение пользователя), Model преобразует его в новое состояние, View отображает состояние. Подход удобен для реактивных интерфейсов и сложных состояний, но требует дисциплины и дополнительной инфраструктуры (RxJava, Kotlin Flow, редьюсеры).

VIPER (View — Interactor — Presenter — Entity — Router)

VIPER дробит модуль на пять компонентов и хорошо подходит для крупных команд, где важна максимальная изоляция обязанностей. Плата за это — большое количество классов и сложность реализации даже для простого экрана; шаблон исторически распространён в iOS.

Сравнение подходов приведено в таблице 1.

Шаблон	Компоненты	Когда применять	Ограничения
MVC	Model, View, Controller	Простые приложения, веб-разработка	Контроллер перегружается, View связана с Model
MVP	Model, View, Presenter	Экраны с большим числом пользовательских действий	Много интерфейсов, Presenter разрастается
MVVM	Model, View, ViewModel	Android-приложения с lifecycle и LiveData	Нужны дисциплина состояний и фабрики зависимостей
MVI	Model, View, Intent	Реактивные интерфейсы, сложные состояния	Требует редьюсеров и управления потоками
VIPER	View, Interactor, Presenter, Entity, Router	Крупные модульные проекты (в основном iOS)	Много компонентов на каждый модуль

Таблица 1 — Сравнение архитектурных шаблонов

Для Cybersport Tracker выбран MVVM. Причины: проект уже разделён на модули app/domain/data, использует AndroidX и Java, а главное — должен сохранять состояние при повороте экрана и реагировать на изменения базы данных. MVVM и LiveData решают эти задачи без написания собственного жизненного цикла и без привязки ViewModel к Activity.

1.2 MVVM и чистая архитектура проекта

В проекте реализована схема «чистой архитектуры» с правилом зависимостей, направленных внутрь: слой представления зависит от domain, слой data реализует интерфейсы domain, а domain не знает ни об Android, ни о Room, ни о JSON. Такое разделение позволяет заменить источник данных, не меняя ViewModel и Activity.

- app (presentation) — Activity, RecyclerView-адаптеры, ViewModel, фабрики ViewModel, UI-состояния, AppContainer как composition root.
- domain — доменные модели Team, Match, Tournament, Prediction, User; интерфейсы TeamRepository, FavoritesRepository, UserRepository и др.; use case-сценарии (GetTeamsUseCase, AddTeamToFavoritesUseCase, GetTeamByIdUseCase и т. д.).

- data — DTO и мапперы, NetworkApi с mock-JSON, Room (сущности, DAO, AppDatabase), SharedPreferences, Firebase Auth, TFLite-источник; реализации репозиториев.

Схема зависимостей и поток данных показаны на рисунке 1.

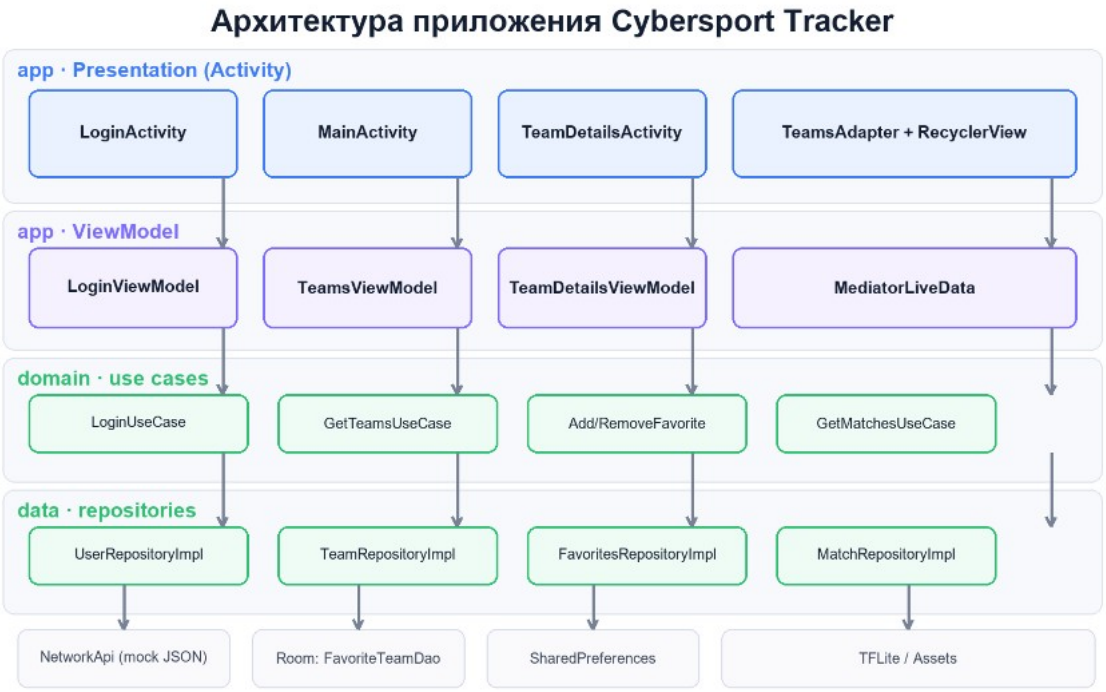


Рисунок 1 — Архитектура приложения Cybersport Tracker

Ключевая идея: Activity не обращается к репозиторию или NetworkApi напрямую. Она создаёт ViewModel через ViewModelProvider и фабрику, а ViewModel вызывает use case. Единственное место, где собираются реализации, — AppContainer; Activity получает из него уже готовые сценарии и наблюдаемые источники.

1.3 ViewModel: жизненный цикл, ViewModelProvider, Factory

ViewModel — компонент AndroidX, предназначенный для хранения и управления данными, связанными со слоем представления. Он привязан к ViewModelStoreOwner (Activity или Fragment) и сохраняется при пересоздании Activity, вызванном поворотом экрана или сменой конфигурации. Когда владелец окончательно уничтожается, вызывается protected-метод onCleared(), в котором освобождаются ресурсы: останавливаются executor-ы, снимаются подписки.

Важно понимать границы: ViewModel не является заменой onSaveInstanceState. Если операционная система уничтожает процесс приложения, ViewModel тоже теряется; для восстановления критичных данных нужны SavedStateHandle или постоянное

хранилище. В рамках лабораторной работы ViewModel решает именно проблему пересоздания Activity при повороте.

Экземпляр ViewModel нельзя создавать оператором new в Activity, если требуется правильное связывание с жизненным циклом. Используется ViewModelProvider:

Листинг 1 — Корректное получение ViewModel

```
vm = new ViewModelProvider(this, new TeamsViewModelFactory(container))
        .get(TeamsViewModel.class);
```

Если у ViewModel есть зависимости, создаётся фабрика, реализующая ViewModelProvider.Factory. Фабрика изолирует Activity от способа конструирования зависимостей и позволяет передать в ViewModel use case-сценарии и LiveData-источники:

Листинг 2 — Фабрика ViewModel

```
public class TeamsViewModelFactory implements ViewModelProvider.Factory {
    private final AppContainer container;

    public TeamsViewModelFactory(AppContainer container) {
        this.container = container;
    }

    @NonNull
    @Override
    @SuppressWarnings("unchecked")
    public <T extends ViewModel> T create(@NonNull Class<T> modelClass) {
        if (modelClass.isAssignableFrom(TeamsViewModel.class)) {
            return (T) new TeamsViewModel(
                    container.teams(),
                    container.addToFavorites(),
                    container.removeFromFavorites(),
                    container.profile(),
                    container.logout(),
                    container.observeFavoriteTeams()
            );
        }
        throw new IllegalArgumentException("Unknown ViewModel class");
    }
}
```

ViewModel не должна хранить ссылки на Activity, View или Context: это приводит к утечкам памяти. В проекте ViewModel работает только с доменными моделями, use case-сценариями и LiveData; Android-контекст в неё не передаётся.

1.4 LiveData и организация реактивного состояния

LiveData — наблюдаемый держатель данных, учитывающий жизненный цикл. Наблюдатель (Observer) получает обновления только тогда, когда LifecycleOwner находится в активном состоянии (STARTED или RESUMED). Это автоматически предотвращает обновление уничтоженной Activity и утечки памяти. При переходе экрана в активное состояние LiveData доставляет последнее установленное значение — поэтому после поворота интерфейс восстанавливается без повторной загрузки данных.

Основные операции:

- setValue() — установка значения из главного потока;
- postValue() — установка значения из фонового потока (значение доставляется на main thread);
- observe(owner, observer) — подписка с учётом жизненного цикла;
- Transformations.map()/switchMap() — преобразование одного LiveData в другое.

В проекте используются два вида LiveData. MutableLiveData применяется внутри ViewModel для изменения состояния, а наружу отдаётся неизменяемый тип LiveData — Activity не может случайно изменить состояние в обход ViewModel. Для разовых событий (сообщение Snackbar, навигация после выхода из аккаунта) реализован SingleLiveEvent, который доставляет значение только один раз и не повторяет его после поворота экрана.

Листинг 3 — SingleLiveEvent для разовых событий

```
public class SingleLiveEvent<T> extends MutableLiveData<T> {
    private final AtomicBoolean pending = new AtomicBoolean(false);

    @Override
    public void setValue(T value) {
        pending.set(true);
        super.setValue(value);
    }

    @Override
    public void postValue(T value) {
        pending.set(true);
        super.postValue(value);
    }

    @Override
    public void observe(@NonNull LifecycleOwner owner,
                        @NonNull Observer<? super T> observer) {
        super.observe(owner, value -> {
            if (pending.compareAndSet(true, false)) {
                observer.onChanged(value);
            }
        });
    }
}
```

1.5 MediatorLiveData как средство объединения источников

MediatorLiveData — наследник LiveData, который может наблюдать за несколькими источниками одновременно. Метод addSource(LiveData<S> source, Observer<S> onChanged) подписывается на источник и вызывает обработчик при каждом его изменении; removeSource() отписывается. Это позволяет собрать несколько потоков данных в одно согласованное состояние.

Типичный сценарий — экран, которому нужны данные из сети и из локальной базы. Если обрабатывать их в Activity двумя отдельными observe-колбэками, легко

получить рассинхронизацию: один колбэк уже обновил список, второй ещё нет. MediatorLiveData переносит объединение в ViewModel: любое изменение любого источника пересчитывает единое состояние TeamsUiState, и Activity получает уже согласованный снимок.

В Cybersport Tracker MediatorLiveData объединяет:

- MutableLiveData<List<Team>> с командами из замоканного сетевого слоя;
- LiveData<List<Team>> из Room с избранными командами;
- MutableLiveData<Boolean> loading, MutableLiveData<String> query и country — служебные состояния загрузки и фильтрации.

Схема объединения представлена на рисунке 2.

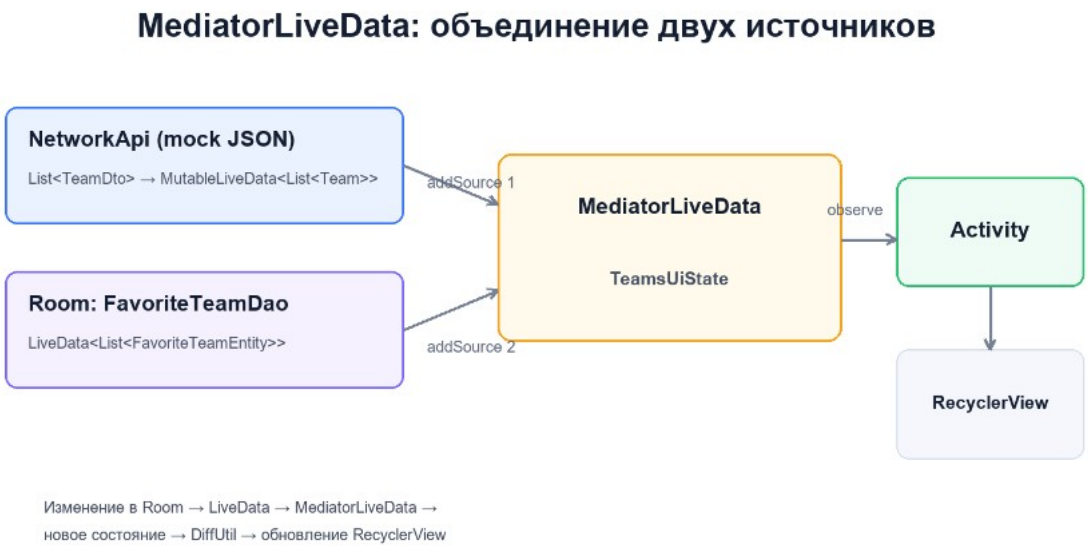


Рисунок 2 — MediatorLiveData: два источника и одно состояние экрана

Когда пользователь нажимает на «сердечко», ViewModel в фоновом executor-е вызывает AddTeamToFavoritesUseCase. Репозиторий пишет строку в Room. Room сам обновляет LiveData, MediatorLiveData пересчитывает TeamsUiState, Activity получает новый список и передаёт его в адаптер. Ручной вызов notifyDataSetChanged() не требуется.

1.6 RecyclerView: Adapter, ViewHolder, LayoutManager, DiffUtil

RecyclerView — компонент для отображения больших наборов данных в виде списка или сетки. В отличие от ListView, он не создаёт View для каждого элемента, а переиспользует уже созданные представления, вышедшие за пределы экрана (view recycling). Это снижает потребление памяти и делает прокрутку плавной.

Основные участники:

- Adapter — посредник между данными и RecyclerView; создаёт ViewHolder и привязывает данные;
- ViewHolder — хранит ссылки на View одного элемента, чтобы не вызывать findViewById() при каждой прокрутке;
- LayoutManager — отвечает за расположение элементов (LinearLayoutManager, GridLayoutManager, StaggeredGridLayoutManager);
- ItemDecoration — разделители и отступы; ItemAnimator — анимации изменений; ItemTouchHelper — drag-and-drop и swipe-to-dismiss.

Adapter обязан переопределить три метода: onCreateViewHolder() (создание ViewHolder из XML-разметки), onBindViewHolder() (привязка данных к позиции) и getItemCount() (количество элементов). В проекте используется ListAdapter — наследник RecyclerView.Adapter, который сам применяет DiffUtil. DiffUtil сравнивает старый и новый списки и сообщает RecyclerView, какие элементы добавлены, удалены или изменены; анимаются только реальные изменения, а не весь список.

Клик по элементу обрабатывается через слушатель, переданный в адаптер; клик по кнопке избранного — отдельный слушатель. Это позволяет Activity оставаться координатором навигации, не «зашивая» её в адаптер.

2 Практическая работа № 3. ViewModel, LiveData, MediatorLiveData

2.1 Контрольное задание и исходное состояние проекта

Контрольное задание практической работы № 3 содержит три требования:

1. взаимодействие Activity со слоем domain должно осуществляться через ViewModel;
2. обновление состояния интерфейса должно осуществляться через LiveData;
3. необходимо самостоятельно изучить MediatorLiveData и получить данные из замоканных данных сетевой части и базы данных.

Исходная версия приложения этим требованиям не удовлетворяла. MainActivity напрямую создавала AppContainer и вызывала container.profile(), container.teams(), container.addToFavorites() и другие сценарии, а результат выводила в один TextView. Такая реализация имела несколько недостатков:

- бизнес-логика и навигация были сосредоточены в Activity (более 200 строк);

- при повороте экрана Activity пересоздавалась и все сценарии выполнялись заново;
- не было единого состояния экрана и реактивного обновления при изменении базы;
- код было сложно тестировать без Android-инструментов.

В ходе работы презентационный слой был переработан: появились LoginViewModel, TeamsViewModel и TeamDetailsViewModel, состояния LoginUiState, TeamsUiState, TeamDetailsUiState, фабрики ViewModel и реактивные источники данных.

2.2 Экранные ViewModel и фабрики

LoginViewModel инкапсулирует вход и регистрацию. Activity передаёт в него введённые e-mail и пароль, а получает LoginUiState с флагами loading, error и user. Проверка пустых полей и обработка ошибок также перенесены во ViewModel; Activity лишь преобразует код ошибки в строковый ресурс.

TeamsViewModel — центральный класс экрана «Команды». Он получает из AppContainer use case-сценарии, профиль пользователя и LiveData избранного. Внутри создаётся single-thread executor для операций с Room и mock-сетевым слоем, чтобы не блокировать главный поток. Наружу ViewModel отдаёт два LiveData: state (единое состояние экрана) и message (разовое сообщение).

Листинг 4 — Поля TeamsViewModel

```
public class TeamsViewModel extends ViewModel {
    private final GetTeamsUseCase getTeamsUseCase;
    private final AddTeamToFavoritesUseCase addTeamToFavoritesUseCase;
    private final RemoveTeamFromFavoritesUseCase removeTeamFromFavoritesUseCase;
    private final LogoutUseCase logoutUseCase;
    private final User user;

    private final ExecutorService executor = Executors.newSingleThreadExecutor();

    private final MutableLiveData<List<Team>> networkTeams =
            new MutableLiveData<>(Collections.emptyList());
    private final LiveData<List<Team>> favoriteTeams;

    private final MutableLiveData<Boolean> loading = new MutableLiveData<>(true);
    private final MutableLiveData<String> query = new MutableLiveData<>("");
    private final MutableLiveData<String> country = new MutableLiveData<>("");
    private final MediatorLiveData<TeamsUiState> state = new MediatorLiveData<>();
    private final SingleLiveEvent<Integer> message = new SingleLiveEvent<>();
    private final SingleLiveEvent<Boolean> loggedOut = new SingleLiveEvent<>();

    private volatile Set<Integer> favoriteIds = Collections.emptySet();
}
```

Загрузка команд выполняется в фоне, а результат публикуется через postValue(). Это принципиально: Room и «сетевой» слой не должны работать в главном потоке.

Листинг 5 — Асинхронная загрузка данных

```
private void loadTeams() {
    executor.execute(() -> {
        List<Team> loaded = getTeamsUseCase.execute();
        networkTeams.postValue(loaded == null
                ? Collections.emptyList()
                : new ArrayList<>(loaded));
        loading.postValue(false);
    });
}
```

TeamDetailsViewModel устроен аналогично, но дополнительно загружает матчи команды и фильтрует их по идентификатору. Его состояние содержит команду, список матчей, признак избранного и признак авторизации. Благодаря этому Activity не выполняет ни одного запроса к domain-слою.

2.3 Обновление интерфейса через LiveData

Activity подписывается на LiveData в onCreate() и передаёт состояние в метод render(). render() не содержит бизнес-логики: он только показывает или скрывает блоки, заполняет текстовые поля и передаёт список в адаптер. После поворота экрана Activity создаётся заново, но ViewModel остаётся той же, и LiveData немедленно отдаёт последнее состояние.

Листинг 6 — Подписка Activity на LiveData

```
viewModel.getState().observe(this, this::render);
viewModel.getMessage().observe(this, messageRes ->
        Snackbar.make(root, messageRes, Snackbar.LENGTH_LONG).show());
viewModel.getLoggedOut().observe(this, loggedOut -> {
    if (Boolean.TRUE.equals(loggedOut)) {
        openLogin();
    }
});
```

Метод render() реализует декларативное отображение состояния:

Листинг 7 — Отрисовка состояния экрана

```
private void render(TeamsUiState state) {
    currentUser = state.getUser();
    adapter.submitList(state.getTeams());

    boolean showList = !state.isLoading() && !state.isEmpty();
    recyclerView.setVisibility(showList ? View.VISIBLE : View.INVISIBLE);
    loadingView.setVisibility(state.isLoading() ? View.VISIBLE : View.GONE);
    emptyView.setVisibility(state.isEmpty() ? View.VISIBLE : View.GONE);
    guestCard.setVisibility(state.isAuthorized() ? View.GONE : View.VISIBLE);

    if (state.isLoading()) {
        countView.setText(R.string.teams_loading);
    } else if (state.hasFilter()) {
        countView.setText(getString(R.string.teams_found, state.getFoundCount()));
    } else {
        countView.setText(getString(R.string.teams_total, state.getTotalCount()));
    }
}
```

Такой подход устраняет ручные вызовы setText() в обработчиках нажатий и гарантирует, что все элементы экрана отображают одно и то же состояние. Например, при поиске одновременно обновляются счётчик, список и пустое состояние.

2.4 Объединение сетевых данных и Room с помощью MediatorLiveData

Для выполнения третьего пункта задания в DAO избранного добавлен метод, возвращающий LiveData. Room поддерживает реактивные запросы: после вставки или удаления строки он сам пересчитывает результат и уведомляет наблюдателей.

Листинг 8 — Реактивный запрос Room

```
@Query("SELECT * FROM favorite_teams ORDER BY worldRank ASC")
LiveData<List<FavoriteTeamEntity>> observeAll();
```

Репозиторий преобразует сущности в доменные модели с помощью Transformations.map, не раскрывая слой data наружу:

Листинг 9 — Репозиторий избранного

```
public LiveData<List<Team>> observeFavoriteTeams() {
    return Transformations.map(favoriteTeamDao.observeAll(),
            FavoriteTeamEntityMapper::toDomainList);
}
```

В конструкторе TeamsViewModel MediatorLiveData получает пять источников. Обработчики не пытаются обновить UI напрямую — каждый из них лишь вызывает emitState(), который пересчитывает единое состояние.

Листинг 10 — Подключение источников к MediatorLiveData

```
state.addSource(networkTeams, teams -> emitState());
state.addSource(loading, isLoading -> emitState());
state.addSource(query, value -> emitState());
state.addSource(country, value -> emitState());
state.addSource(favoriteTeams, teams -> {
    Set<Integer> ids = new HashSet<>();
    if (teams != null) {
        for (Team team : teams) {
            ids.add(team.getId());
        }
    }
    favoriteIds = ids;
    emitState();
});
```

Метод emitState() применяет поиск и фильтр, добавляет признак избранного и формирует неизменяемый TeamsUiState:

Листинг 11 — Формирование единого состояния

```
List<TeamListItem> visible = new ArrayList<>();
for (Team team : all) {
    if (!matchesCountry(team, currentCountry)) {
        continue;
    }
    if (!matchesQuery(team, currentQuery)) {
        continue;
    }
    visible.add(new TeamListItem(
            team,
            TeamLogoResolver.resolve(team),
            user != null && favoriteIds.contains(team.getId()),
            user != null
    ));
}
state.setValue(new TeamsUiState(
        isLoading, user, visible, all.size(), visible.size(),
        currentQuery, currentCountry, !isLoading && visible.isEmpty()
));
```

Переключение избранного также выполняется в фоновом потоке. После записи в Room срабатывает observeAll(), MediatorLiveData пересчитывает состояние, и сердечко меняет цвет без перезагрузки экрана и без вызова notifyDataSetChanged():

Листинг 12 — Переключение избранного

```
public void toggleFavorite(Team team) {
    if (team == null) {
        return;
    }
    if (user == null) {
        message.setValue(R.string.teams_login_hint);
        return;
    }
    executor.execute(() -> {
        if (favoriteIds.contains(team.getId())) {
            removeTeamFromFavoritesUseCase.execute(team.getId());
        } else {
            addTeamToFavoritesUseCase.execute(team);
        }
    });
}
```

Аналогичный MediatorLiveData используется в TeamDetailsViewModel: он объединяет команду, её матчи, флаг загрузки и наблюдаемое избранное. Это демонстрирует, что подход масштабируется на несколько экранов, а не является единичным трюком.

2.5 Результаты практической работы № 3

После рефакторинга Activity перестала обращаться к use case-сценариям напрямую. Взаимодействие с domain-слоем идёт только через ViewModel, состояние интерфейса обновляется через LiveData, а MediatorLiveData объединяет mock-сеть и Room. На рисунках 3–6 показаны основные экраны приложения.

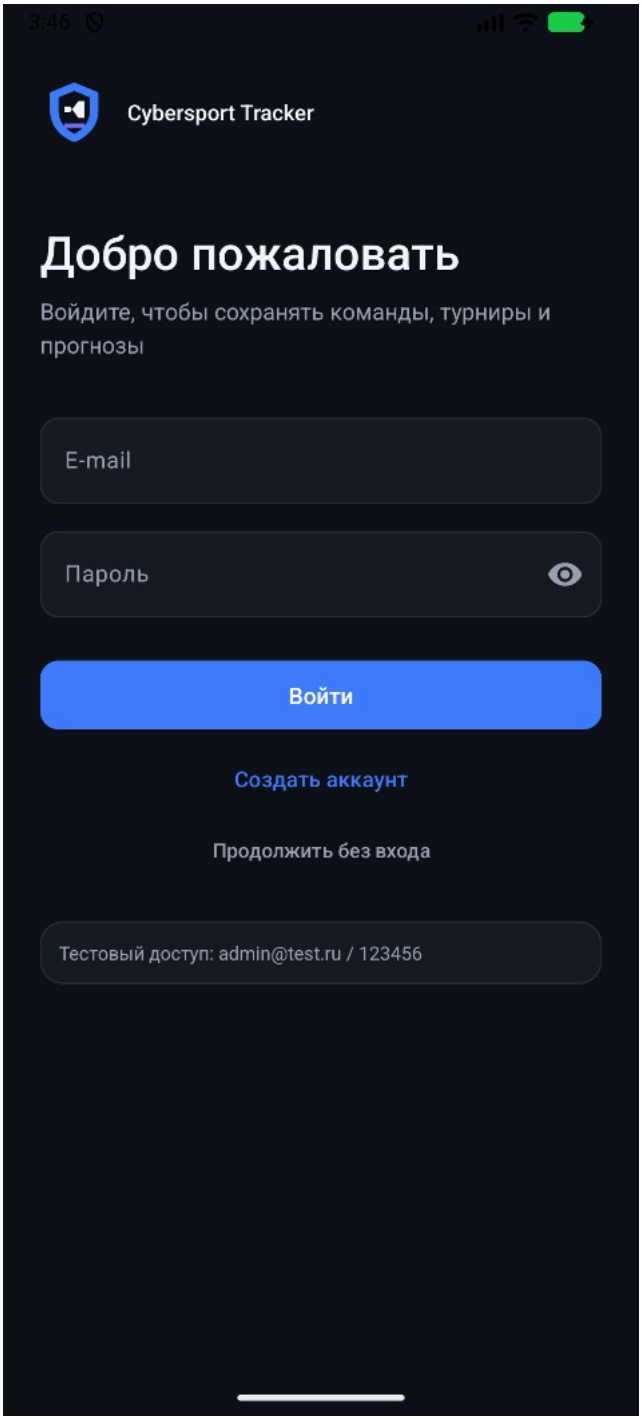


Рисунок 3 — Экран авторизации: тёмная тема, поля Material 3

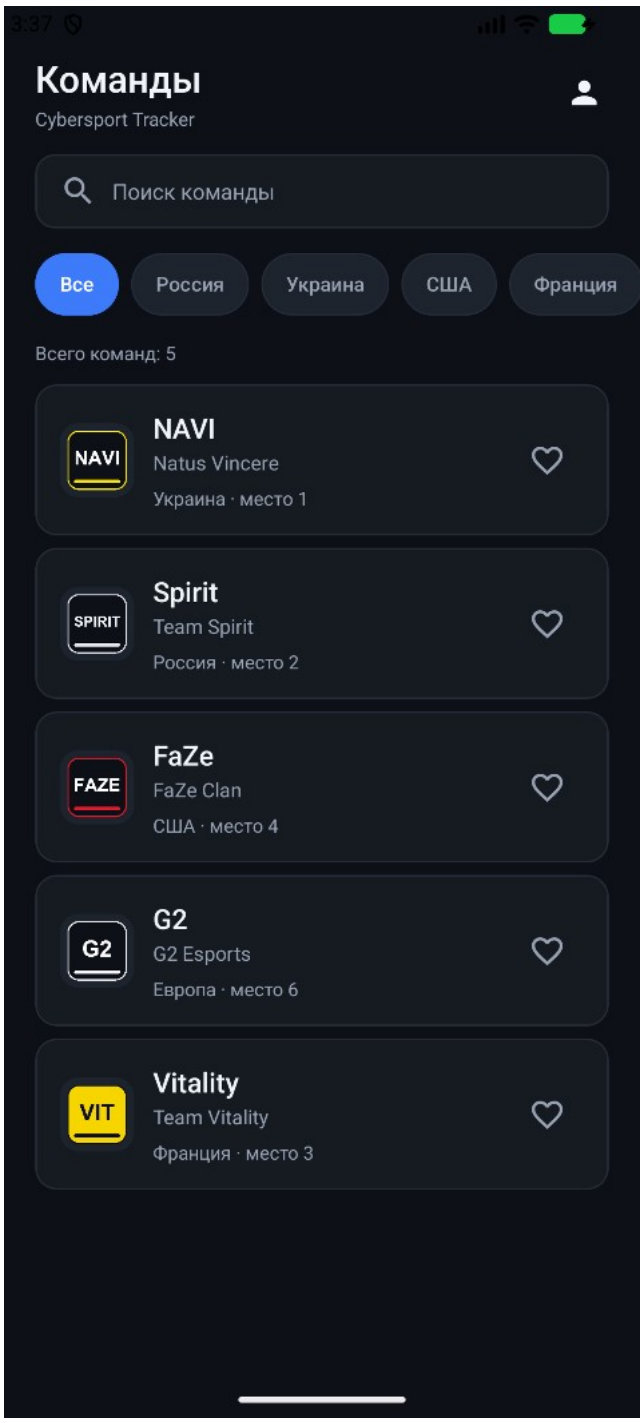


Рисунок 4 — Экран «Команды» после входа: данные из mock-слоя

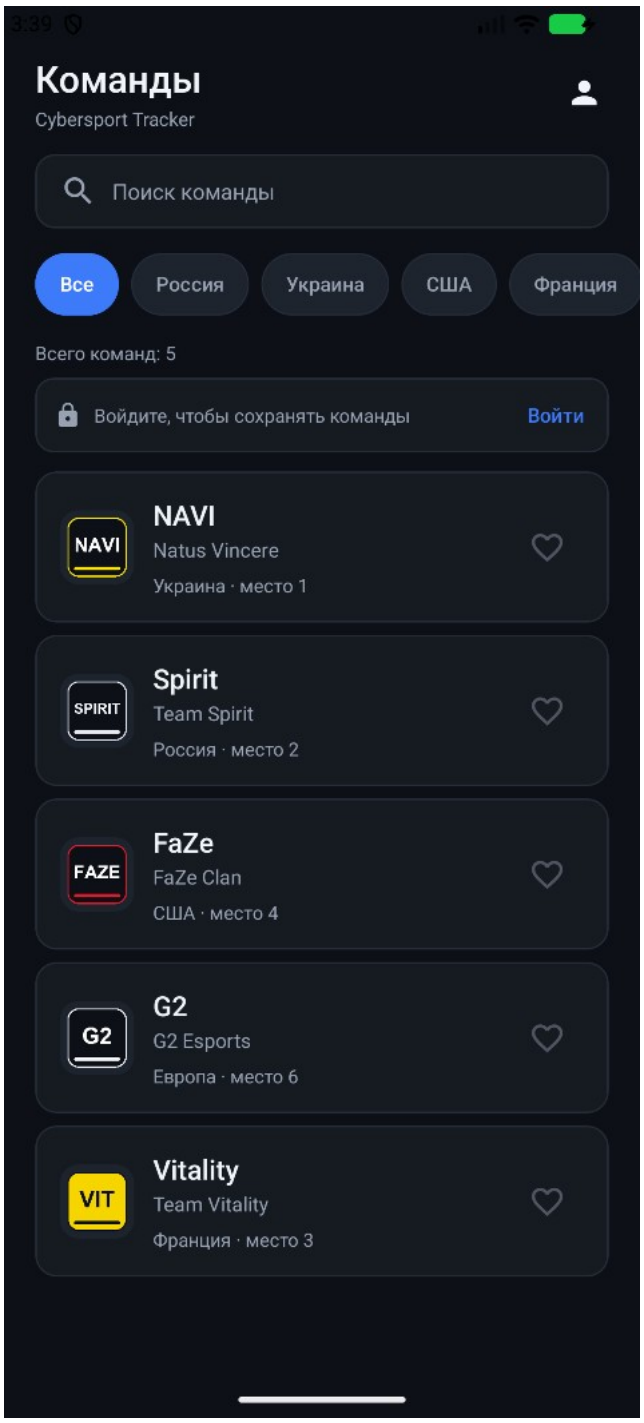


Рисунок 5 — Гостевой режим: подсказка и неактивные действия избранного

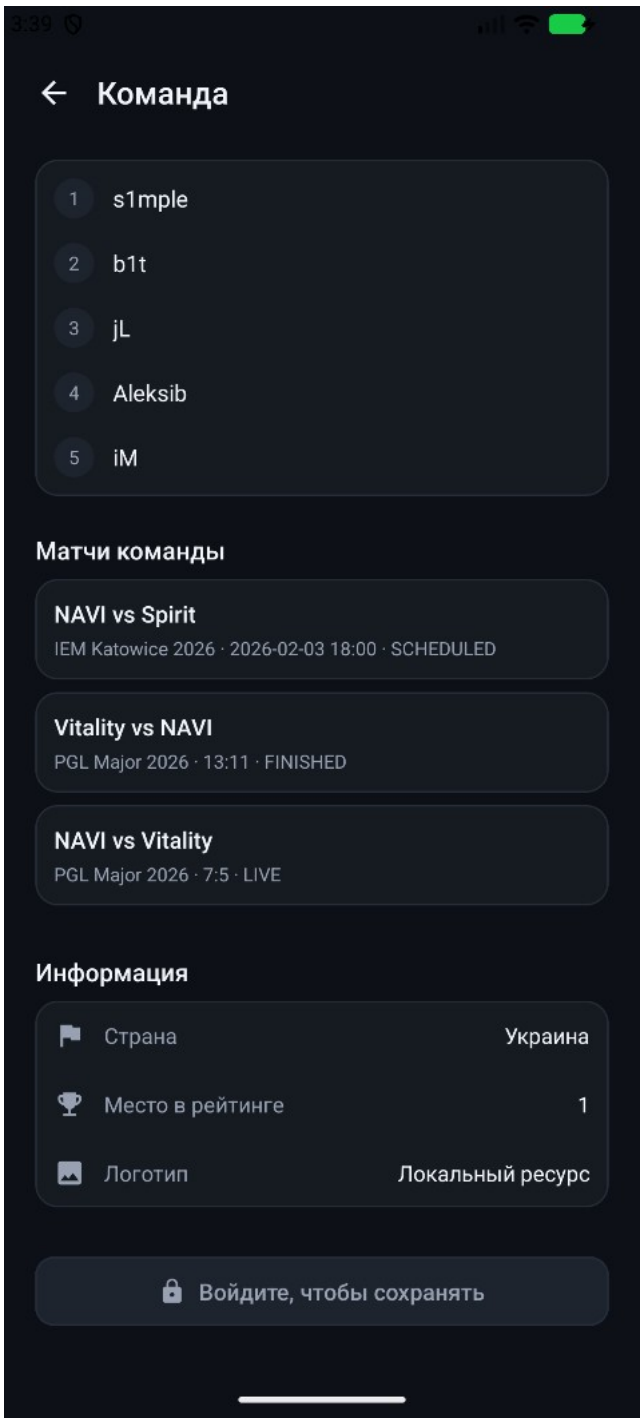


Рисунок 6 — Карточка команды в гостевом режиме: заблокированная кнопка

3 Практическая работа № 4. RecyclerView и передача данных через LiveData

3.1 Контрольное задание

Контрольное задание практической работы № 4 требует открыть приложение из практического занятия № 1 и изменить его по аналогии с примером: создать в репозитории заглушку с набором данных, которые планируется получать от внешнего

сервиса, передать эти данные в слой представления с помощью LiveData и установить в RecyclerView.

В проекте Cybersport Tracker такая заглушка уже существовала в виде NetworkApi, возвращающего JSON-строки. В ходе работы она была дополнена и явно отделена от реализации: DTO → маппер → доменная модель → use case → ViewModel → LiveData → RecyclerView. Это позволяет позже заменить NetworkApi на Retrofit, не меняя UI.

3.2 Заглушка набора данных в слое data

NetworkApi содержит константы TEAMS_JSON, TOURNAMENTS_JSON, MATCHES_JSON и USERS_JSON. Gson разбирает их в DTO, а TeamMapper преобразует TeamDto в доменную модель Team. Пять команд соответствуют реалистичному контенту из технического задания:

Листинг 13 — Фрагмент заглушки сетевого ответа

```
private static final String TEAMS_JSON = "["
        + "{\"id\":1,\"tag\":\"NAVI\",\"name\":\"Natus Vincere\","
        + "\"country\":\"Украина\",\"logo_url\":\"https://mockapi.io/logos/navi.png\","
        + "\"world_rank\":1,\"players\":[\"s1mple\",\"b1t\",\"jL\",\"Aleksib\",\"iM\"]},"
        + "{\"id\":2,\"tag\":\"Spirit\",\"name\":\"Team Spirit\","
        + "\"country\":\"Россия\",\"world_rank\":2, ...}"
        + "]";
```

TeamRepositoryImpl реализует интерфейс domain-слоя и возвращает List<Team>. GetTeamsUseCase инкапсулирует вызов репозитория, а ViewModel загружает данные в фоновом потоке и публикует их в MutableLiveData. Таким образом, источник данных остаётся заменяемым: Activity и адаптер не знают, откуда пришли команды.

3.3 Реализация списка команд на RecyclerView

Разметка экрана содержит RecyclerView, который занимает всё доступное пространство между панелью поиска и нижней границей. Элемент списка — MaterialCardView с ConstraintLayout внутри: логотип 48 dp в скруглённом контейнере, тег команды, полное название, страна и место в рейтинге, а также кнопка избранного 48 dp. Размеры и отступы соответствуют 8-pt системе, радиус карточки — 14 dp, цвет поверхности — #161B22.

Листинг 14 — Фрагмент разметки элемента списка

```
<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:cardBackgroundColor="@color/surface"
    app:cardCornerRadius="14dp"
    app:cardElevation="0dp"
    app:strokeColor="@color/border"
    app:strokeWidth="1dp">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:padding="16dp">

        <ImageView android:id="@+id/imageTeamLogo"
            android:layout_width="48dp"
            android:layout_height="48dp"
            android:background="@drawable/bg_logo_tile"
            android:padding="4dp" />
        <!-- tag, name, meta, favorite -->
    </androidx.constraintlayout.widget.ConstraintLayout>
</com.google.android.material.card.MaterialCardView>
```

TeamsAdapter наследуется от ListAdapter и использует DiffUtil. Внутренний TeamViewHolder хранит ссылки на ImageView, три TextView и ImageButton; метод bind() заполняет их и навешивает обработчики кликов. Логотип выбирается через TeamLogoResolver по идентификатору команды: в офлайн-демонстрации используются локальные PNG-ресурсы, а в production здесь была бы загрузка по logoUrl.

Листинг 15 — Адаптер списка команд

```
public class TeamsAdapter extends ListAdapter<TeamListItem, TeamsAdapter.TeamViewHolder> {

    private static final DiffUtil.ItemCallback<TeamListItem> DIFF_CALLBACK =
            new DiffUtil.ItemCallback<TeamListItem>() {
                @Override
                public boolean areItemsTheSame(TeamListItem oldItem, TeamListItem newItem) {
                    return oldItem.getTeam().getId() == newItem.getTeam().getId();
                }

                @Override
                public boolean areContentsTheSame(TeamListItem oldItem, TeamListItem newItem) {
                    return oldItem.equals(newItem);
                }
            };

    public TeamsAdapter(Listener listener) {
        super(DIFF_CALLBACK);
        this.listener = listener;
    }

    @NonNull
    @Override
    public TeamViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
        View view = LayoutInflater.from(parent.getContext())
                .inflate(R.layout.item_team, parent, false);
        return new TeamViewHolder(view, listener);
    }

    @Override
    public void onBindViewHolder(@NonNull TeamViewHolder holder, int position) {
        holder.bind(getItem(position));
    }
}
```

Каждый элемент получает два обработчика: onTeamClick() открывает TeamDetailsActivity с идентификатором команды, onFavoriteClick() передаёт команду в ViewModel. DiffUtil сравнивает элементы по id и содержимому, поэтому после обновления избранного перерисовывается только изменённая карточка.

3.4 Передача данных из ViewModel в адаптер

Цепочка передачи данных выглядит так:

1. NetworkApi возвращает mock-JSON;
2. TeamRepositoryImpl и GetTeamsUseCase преобразуют его в List<Team>;
3. TeamsViewModel загружает список в executor и публикует в MutableLiveData;
4. MediatorLiveData объединяет команды с избранным и формирует List<TeamListItem>;
5. Activity получает TeamsUiState через observe();
6. adapter.submitList(state.getTeams()) передаёт список в ListAdapter;
7. DiffUtil вычисляет различия, RecyclerView перерисовывает только изменившиеся элементы.

Адаптер не подписывается на LiveData самостоятельно: это нарушило бы жизненный цикл и усложнило бы повторное использование. Activity является владельцем жизненного цикла и передаёт в адаптер уже готовый неизменяемый список. Такой подход соответствует рекомендациям Android и упрощает тестирование.

3.5 Состояния списка и результаты работы

Экран поддерживает несколько состояний, которые переключаются через TeamsUiState:

- загрузка — CircularProgressIndicator и подпись «Загрузка команд…»;
- список — карточки команд с логотипами и избранным;
- пустой результат — иконка, заголовок «Команды не найдены» и подсказка изменить запрос;
- гостевой режим — карточка-подсказка «Войдите, чтобы сохранять команды» и приглушённые кнопки избранного;
- поиск и фильтрация по стране — счётчик «Найдено команд: N» и отфильтрованный список.

На рисунках 7–10 показаны состояния списка и карточка команды.

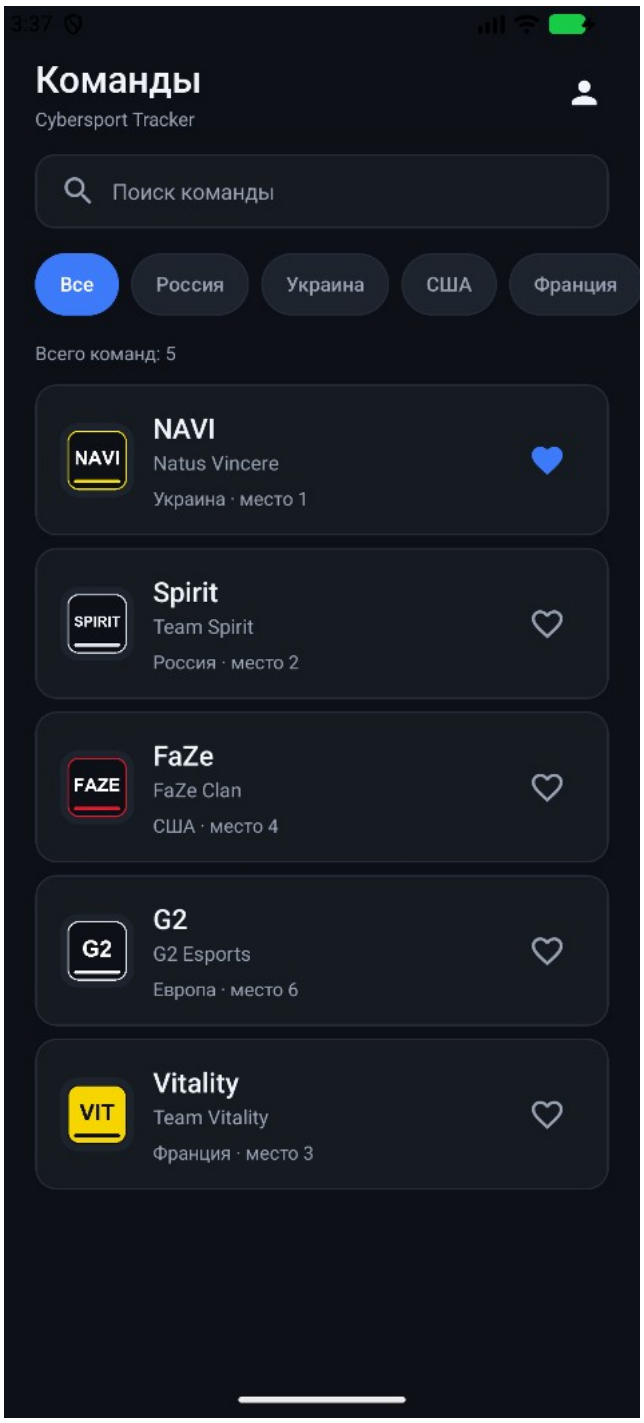


Рисунок 7 — Избранное: сердечко NAVI обновилось через Room LiveData

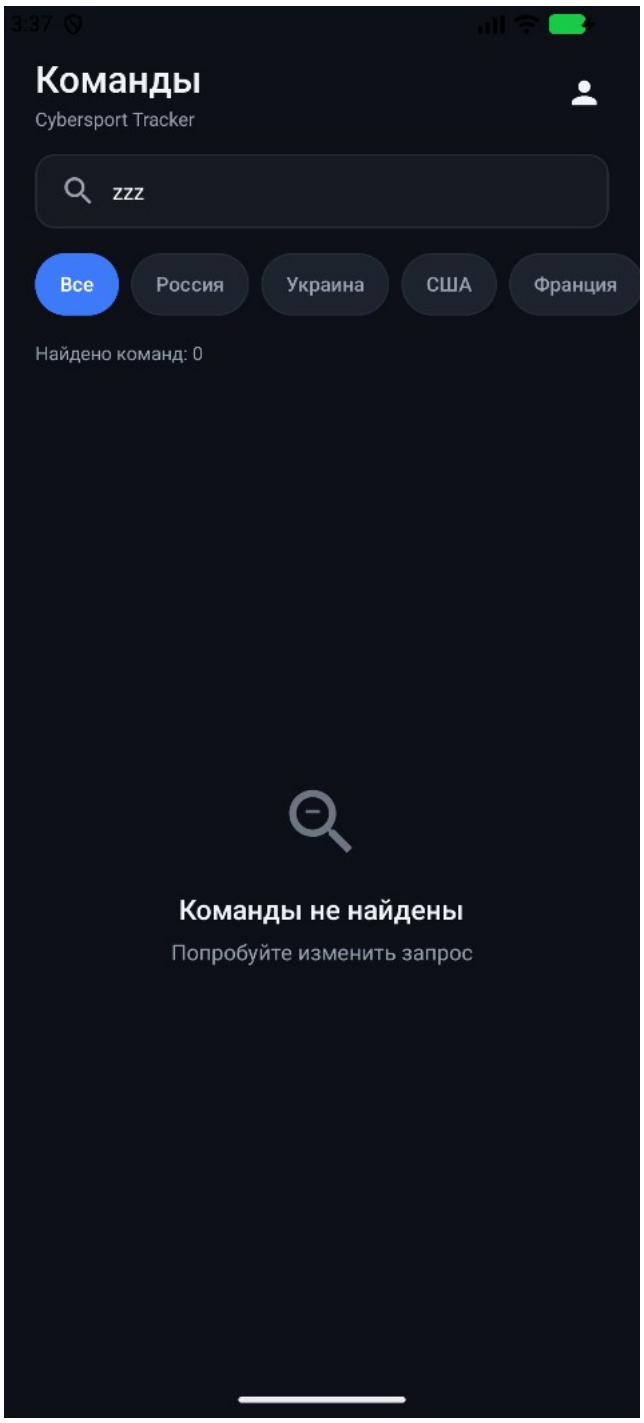


Рисунок 8 — Пустое состояние поиска



Рисунок 9 — Карточка команды: состав и матчи

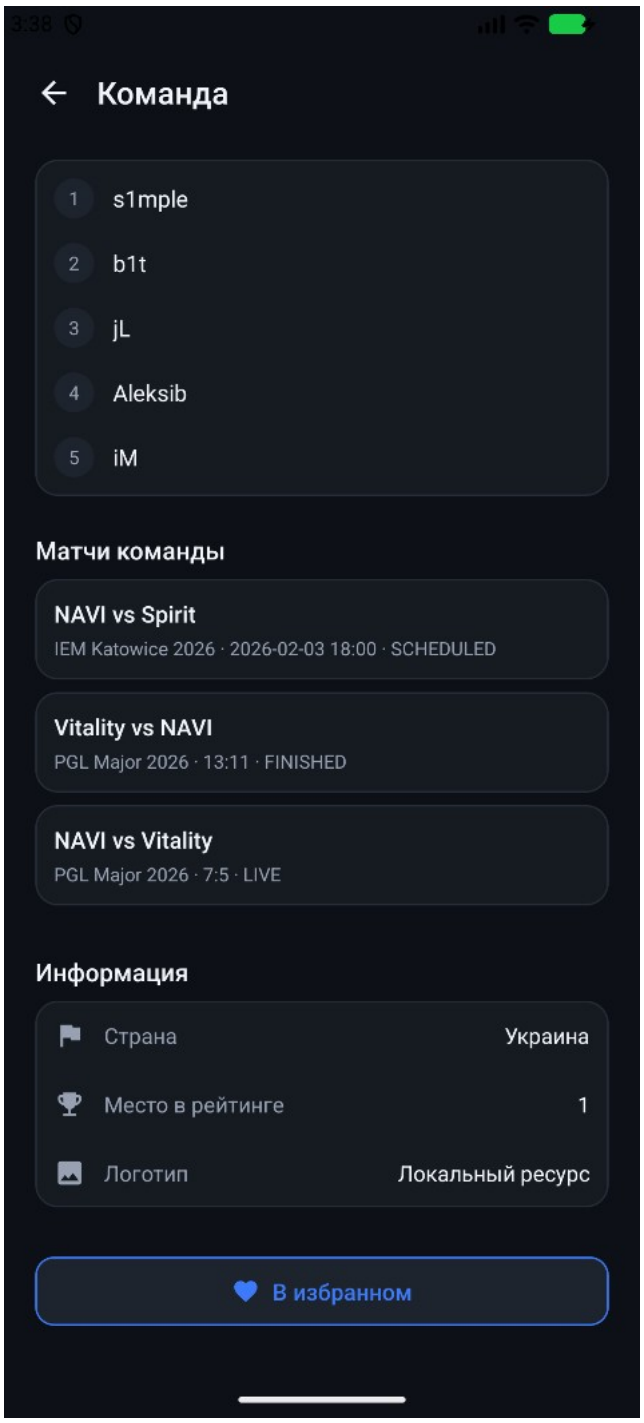


Рисунок 10 — Нижняя часть карточки: информация и кнопка избранного

4 Тестирование и проверка работоспособности

Проверка выполнялась на эмуляторе Medium_Phone_API_36.1 (Android 16, 1080×2400). Проект собирался командой assembleDebug; APK устанавливался через adb. Результаты сценариев приведены в таблице 2.

№	Сценарий	Ожидаемый результат	Факт
1	Сборка проекта (assembleDebug)	BUILD SUCCESSFUL, APK создан	Выполнено
2	Вход admin@test.ru / 123456	Переход на экран «Команды»	Выполнено
3	Загрузка списка	5 команд с логотипами и рейтингом	Выполнено
4	Нажатие на сердечко NAVI	Запись в Room, сердечко синее без перезагрузки	Выполнено
5	Поиск «zzz»	Счётчик 0, пустое состояние	Выполнено
6	Фильтр «Россия»	Одна команда Team Spirit	Выполнено
7	Открытие карточки команды	Состав, матчи, информация, кнопка избранного	Выполнено
8	Выход и «Продолжить без входа»	Гостевая подсказка, заблокированное избранное	Выполнено
9	Поворот экрана	Фильтр и состояние сохраняются благодаря ViewModel	Выполнено

Таблица 2 — Результаты проверки сценариев

Отдельно проверено, что после поворота экрана выбранный фильтр «Россия» и гостевой режим сохраняются: Activity создаётся заново, но ViewModel остаётся той же, а LiveData немедленно отдаёт последнее состояние. Это подтверждает корректность выбранной архитектуры.

Ограничения учебной реализации: сетевой слой является заглушкой и не выполняет реальных HTTP-запросов; логотипы команд локальные; авторизация использует mock-провайдер при недоступности Firebase. Эти ограничения не влияют на демонстрацию архитектурных решений и могут быть устранены заменой реализаций в слое data.

Заключение

В ходе выполнения практических работ № 3 и № 4 приложение Cybersport Tracker было переведено на архитектуру MVVM. Activity больше не содержит бизнес-логики и не обращается к репозиториям напрямую: взаимодействие со слоем domain инкапсулировано

в LoginViewModel, TeamsViewModel и TeamDetailsViewModel, а состояние интерфейса передаётся через LiveData.

Освоен MediatorLiveData: на экране «Команды» он объединяет замоканные данные сетевого слоя и реактивный запрос Room к избранному. Любое изменение базы автоматически пересчитывает единое состояние, поэтому интерфейс обновляется без ручных уведомлений адаптера. Это устраняет рассинхронизацию между списком, счётчиком и кнопками.

Для отображения списка команд реализован RecyclerView с ListAdapter и DiffUtil: ViewHolder переиспользуются, изображения логотипов выводятся через ImageView, а изменения списка анимируются точечно. Передача данных выстроена в цепочку «заглушка → репозиторий → use case → ViewModel → LiveData → Activity → адаптер».

Полученный код собирается, устанавливается на эмулятор и проходит проверку основных сценариев: авторизация, гостевой режим, избранное, поиск, фильтрация, карточка команды и поворот экрана. Архитектура готова к дальнейшему развитию: замене mock-API на Retrofit, добавлению новых экранов и модульному тестированию ViewModel.

Список использованных источников

1. Android Developers. ViewModel overview. — URL: https://developer.android.com/topic/libraries/architecture/viewmodel (дата обращения: 02.11.2026).
2. Android Developers. LiveData overview. — URL: https://developer.android.com/topic/libraries/architecture/livedata (дата обращения: 02.11.2026).
3. Android Developers. MediatorLiveData. — URL: https://developer.android.com/reference/androidx/lifecycle/MediatorLiveData (дата обращения: 02.11.2026).
4. Android Developers. RecyclerView. — URL: https://developer.android.com/develop/ui/views/layout/recyclerview (дата обращения: 02.11.2026).
5. Android Developers. Save data in a local database using Room. — URL: https://developer.android.com/training/data-storage/room (дата обращения: 02.11.2026).
6. Android Developers. Guide to app architecture. — URL: https://developer.android.com/topic/architecture (дата обращения: 02.11.2026).
7. Material Design 3. Components. — URL: https://m3.material.io/components (дата обращения: 02.11.2026).
8. Gradle. Android plugin for Gradle. — URL: https://developer.android.com/build (дата обращения: 02.11.2026).

Приложение А. Перечень изменённых файлов

В таблице А.1 перечислены основные файлы, созданные или изменённые в ходе работ.

Модуль	Файл	Назначение
gradle	libs.versions.toml, app/build.gradle.kts, data/build.gradle.kts	Зависимости lifecycle, RecyclerView, CardView
domain	RemoveTeamFromFavoritesUseCase.java, LogoutUseCase.java	Новые сценарии использования
data	FavoriteTeamDao.java, FavoritesRepositoryImpl.java	Реактивный запрос Room и LiveData избранного
data	NetworkApi.java, CompositeAuthDataSource.java	Заглушка данных и устойчивая авторизация
app	TeamsViewModel.java, TeamsViewModelFactory.java, TeamsUiState.java, TeamListItem.java	Состояние списка, MediatorLiveData, фильтрация
app	TeamDetailsViewModel.java, TeamDetailsViewModelFactory.java, TeamDetailsUiState.java	Состояние карточки команды
app	LoginViewModel.java, LoginViewModelFactory.java, LoginUiState.java	Состояние авторизации
app	TeamsAdapter.java, TeamLogoResolver.java, SingleLiveEvent.java	RecyclerView-адаптер, локальные логотипы, разовые события
app	MainActivity.java, LoginActivity.java, TeamDetailsActivity.java	Экраны и подписка на LiveData
app	activity_main.xml, activity_login.xml, activity_team_details.xml, item_team.xml, item_player.xml	Разметка в стиле тёмного broadcast-интерфейса
app	colors.xml, themes.xml, dimens.xml, drawable/*	Палитра, типографика, иконки, логотипы

Таблица А.1 — Основные изменения в проекте